SLIPER

Hesaplar, Reçete Önerisi ve Kod Rehberi

Bu rehber programın yaptığı her hesabı sırayla anlatır: sensörden gelen ham sayının nasıl basınca ve hıza dönüştüğünü, A–B ve a–b değerlerinin ne anlama geldiğini, pompa basıncının nasıl tahmin edildiğini ve en önemlisi **"akışkanlaştırıcı ekleyin, ince kum ekleyin, iri agregayı azaltın"** gibi reçete önerilerinin hangi mantıkla verildiğini.

Her konuda dört soru cevaplanır: **Ne oluyor?** — **Formül** — **Örnek sayı** — **Kodda nerede?** Örnek sayıların hepsini hesap makinesiyle kontrol edebilirsiniz.

> **Nasıl okunur?**
>
> Hızlı okumak isterseniz: **Bölüm 1** (büyük resim), **Bölüm 2** (temel kavramlar) ve **Bölüm 5** (reçete önerisi) yeterlidir. Bölüm 3–4 hesapları adım adım açar, Bölüm 6–7 kod haritası ve ayar sayılarıdır.

Kaynak kod sürümü: commit 7ec2d48 ("Öneri sunma yapıldı"), 04.11.2026.

İçindekiler

1. Büyük resim **3**

1.1 Cihaz tek cümlede ne yapar? . . . 3

1.2 Bir sayının yolculuğu . . . 3

2. Temel kavramlar (formülden önce) **4**

3. Ölçüm hesapları adım adım **5**

3.1 Ham sayı → kg ve mm (kalibrasyon) . . . 5

3.2 kg → basınç (mbar) . . . 5

3.3 Stroke'u yakalama ve "Hazırla" (P0l) . . . 6

3.4 Bir stroke'tan iki sayı: p ve Q . . . 6

3.5 Hatalı stroke ve otomatik eleme . . . 7

3.6 Bütün stroke'lardan bir doğru: A, B ve R^2 . . . 7

3.7 Ölçüm kalite uyarıları . . . 8

4. Pompa basıncı tahmini **10**

4.1 A ve B'yi boyuttan bağımsız yapmak: a ve b . . . 10

4.2 Gerçek pompa hattının basıncı . . . 10

4.3 Tahmin grafiği . . . 11

4.4 Karar: pompalanabilir mi? . . . 11

5. Reçete önerisi: "akışkanlaştırıcı / kum / agrega" nereden geliyor? **13**

5.1 Öneri ne zaman çıkar? . . . 13

5.2 Karar mantığı (üç dal) . . . 13

5.3 Her önerinin arkasındaki fizik . . . 13

5.4 Üç çalışılmış örnek . . . 14

5.5 Bilinmesi gereken sınırlar . . . 15

5.6 Öneri kodun neresinde? . . . 15

6. Kod haritası **16**

6.1 Dosyalar . . . 16

6.2 SliperModel fonksiyonları tek tabloda . . . 16

7. Ayar sayıları (SliperModel.h) **17**

Sözlük . . . 17

1. Büyük resim

1.1 Cihaz tek cümlede ne yapar?

İçi taze beton dolu bir boru, kendi ağırlığıyla sabit bir pistonun üzerinden aşağı kayar. Program iki şeyi ölçer: boru **ne kadar hızlı** kayıyor ve beton pistona **ne kadar basınç** yapıyor. Boruya ek ağırlık koyup kaydırmayı tekrarlayınca "hız arttıkça basınç ne kadar artıyor" ilişkisi ortaya çıkar. Bu ilişkiden, aynı betonun şantiyedeki gerçek pompa hattında kaç bar basınç isteyeceği tahmin edilir. Pompa yetmiyorsa program, betonun reçetesinin hangi yönde değişmesi gerektiğini söyler.

> **Neden bu düzenek pompayı taklit eder?**
>
> Beton pompa hattında borunun ortasında "tıkaç" gibi bütün halinde ilerler; asıl kayma boru duvarına yapışık ince bir **çimento hamuru / harç tabakasında** olur (yağlama tabakası). SLIPER bu tabakayı ölçer: sürgülü boru, pompa borusunun küçük bir parçasını taklit eder. Bu yüzden ölçülen değerler doğrudan pompa hattına taşınabilir.

1.2 Bir sayının yolculuğu

Adım	Ne olur?	Sonuç	Kod
1	ESP32 load cell'i ve mesafe sensörünü okur, Wi-Fi ile gönderir (saniyede 50 paket)	ham sayılar	`esp32/…/sliper_esp32.ino`
2	Ham sayı kalibrasyon tablosuyla kg'a ve mm'ye çevrilir, titreşim süzülür	kg, mm	`WifiManager::jsonSatiriIsle`
3	kg, boru kesitine bölünerek basınca çevrilir; dara çıkarılır	mbar	`SliperModel::agirliktanBasincMbar`
4	Konuma bakılarak stroke (bir kayma) başı ve sonu yakalanır	stroke örnekleri	`Calculator::konumGuncelle`
5	Her stroke'tan basınç p ve debi Q çıkarılır	(Q, p) noktası	`Calculator::strokuBitir`
6	Tüm (Q, p) noktalarına en uygun doğru çizilir	A, B, R^2	`SliperModel::pqDogrusu`
7	A ve B boru boyutundan bağımsız hale getirilir	a, b	`SliperModel::schleibingerA/B`
8	a, b ile gerçek pompa hattının basıncı tahmin edilir	bar	`SliperModel::basincTahmini`
9	Tahmin pompa kapasitesiyle karşılaştırılır; aşılırsa reçete önerisi	karar + öneri	`SliperModel::pompaSorunuNedeni`

Altın kural: Bütün formüller ve ayar sayıları tek bir yerde durur: `src/SliperModel.h` ve `src/SliperModel.cpp`. Diğer dosyalar yalnızca veriyi taşır, kaydeder veya ekrana koyar. Bir hesabı merak ettiğinizde önce oraya bakın.

2. Temel kavramlar (formülden önce)

Formüllerin hepsi birkaç basit fikre dayanır. Bunlar otururs a geri kalanı kolaydır.

Kavram	Anlamı	Günlük hayattan benzetme
Basınç (mbar, bar)	Birim alana düşen kuvvet. 1 bar = 1000 mbar. Pompa basıncı genelde bar, SLIPER ölçümü mbar düzeyindedir.	Aynı ağırlık ince topukla sert, düz tabanla yumuşak basar.
Debi Q (m^3/h)	Saatte borudan geçen beton hacmi. Pompanın "hızı".	Musluktan saatte kaç kova su aktığı.
Stroke	Borunun bir kez yukarıdan aşağı kayması. Her stroke bir ölçüm noktasıdır.	Bir deneme atışı.
Akma direnci (A → a)	Betonu **harekete geçirmek** için gereken basınç. Hız sıfıra yakınken bile vardır.	Ketçap şişesi: önce bir eşiği aşmanız gerekir, sonra akar.
Viskoz direnç (B → b)	**Hız arttıkça** eklenen basınç. Hızlı pompalamada büyür.	Balı hızlı karıştırmak yavaş karıştırmaktan çok daha zordur.
Yağlama tabakası	Boru duvarındaki ince harç tabakası. a ve b aslında bu tabakanın özellikleridir.	Kızağın altındaki kar tabakası.
R^2	Noktaların doğruya ne kadar oturduğu (1 = kusursuz, 0,85 altı şüpheli).	Atışların hedef çizgisi etrafında ne kadar toplu olduğu.

Bütün tahmin şu cümleye dayanır:

Gereken basınç = akma payı + hız payı + yükseklik payı

- **Akma payı** betonun "sertliğinden" gelir; debiye bağlı değildir.
- **Hız payı** betonun "yapışkanlığından" gelir; debi ile doğru orantılı büyür.
- **Yükseklik payı** betonu yukarı kaldırmanın bedelidir; reçeteyle değişmez.

Reçete önerisinin bütün mantığı, bu üç paydan **hangisinin baskın olduğuna** bakmaktır (Bölüm 5).

3. Ölçüm hesapları adım adım

Örneklerde Schleibinger kılavuzundaki gerçek bir ölçüm kullanılmış, sayılar **bizim 128 mm borumuza** göre hesaplanmıştır. Cihaz sabitleri: boru iç çapı `BORU_CAPI_M = 0.128`, dolum yüksekliği `BORU_UZUNLUGU_M = 0.5`.

3.1 Ham sayı → kg ve mm (kalibrasyon)

Ne oluyor? Sensörün gönderdiği ham sayının birimi yoktur. Kalibrasyonda "ham sayı 84 000 iken üstünde 5 kg vardı" gibi noktalar kaydedersiniz. Program iki kayıtlı nokta arasını düz çizgiyle birleştirir ve her ham sayıyı bu çizgiden okur (doğrusal ara değer). Tablonun dışına taşan değerlerde en uçtaki iki noktanın çizgisi uzatılır.

$$\text{gerçek} = \text{sol_gerçek} + (\text{ham} - \text{sol_ham}) \times (\text{sağ_gerçek} - \text{sol_gerçek}) \div (\text{sağ_ham} - \text{sol_ham})$$

Örnek: 80 000 → 0 kg ve 120 000 → 10 kg kayıtlıysa, ham 100 000 → 0 + 20 000 × 10 ÷ 40 000 = **5 kg**.

Mesafe sensörü (TOF250) için kalibrasyon yoksa ham değer santimetre kabul edilip 10 ile çarpılır (ham 54 → 540 mm).

Titreşim süzgeci: Her yeni okuma bir önceki süzülmüş değerle harmanlanır: konumda yeni değerin %35'i (`KONUM_FILTRE_KATSAYISI`), ağırlıkta %40'ı (`AGIRLIK_FILTRE_KATSAYISI`) alınır. Böylece tek bir sıçrayan okuma ekranı oynatmaz.

Kodda nerede?

`SliperModel.cpp` → `kalibrasyonTablosundanOku()` (satır 66) — hesabın kendisi.
`SliperModel.h` → `konumFiltrele()`, `agirlikFiltrele()` — süzgeçler.
`WifiManager.cpp` → `jsonSatiriIsle()` (satır 331) — her paket geldiğinde bunları sırayla çağırır.

3.2 kg → basınç (mbar)

Ne oluyor? Load cell betonun pistona bastığı ağırlığı ölçer. Basınç = kuvvet ÷ alan olduğundan ağırlık, borunun kesit alanına bölünür.

$$\text{boru alanı} = \pi \times D^2 \div 4 = \pi \times 0{,}128^2 \div 4 = 0{,}01287\ \text{m}^2$$

$$\text{basınç (mbar)} = \text{kütle (kg)} \times 9{,}80665 \div \text{boru alanı} \div 100$$

Sonuç: 1 kg = **7,62 mbar**. Load cell 20 kg gösteriyorsa basınç 20 × 7,62 = **152 mbar**. (÷100, Pa'yı mbar'a çevirir: 1 mbar = 100 Pa.)

Boru dururken bile load cell betonun kendi ağırlığını taşır (≈15 kg ≈ 120–155 mbar). Bu yüzden ekranda basınç hiç sıfır olmaz — normaldir, aşağıda P0l olarak çıkarılır.

Dara: "Dara Al" düğmesi o anki basıncı kaydeder ve sonraki her okumadan çıkarır. Yanlışlıkla beton doluyken dara alınmasın diye en fazla 2 kg (15,2 mbar) kabul edilir (`DARA_UST_SINIRI_KG`).

Kodda nerede?

`SliperModel.h` → `agirliktanBasincMbar()` (satır 34).
`WifiManager.cpp` → `daraAl()` (satır 105).

3.3 Stroke'u yakalama ve "Hazırla" (P0l)

Ne oluyor? Program konuma bakarak borunun nerede olduğunu anlar. Üst ve alt konumu Kalibrasyon → Stroke Konumları ekranında siz kaydedersiniz; sensör tepede olduğu için boru indikçe mesafe **artar**. Program yönü bu iki değerden kendi anlar.

Durum	Anlamı
YUKARIDA (yeşil)	Boru üstte. Son 4 saniyelik örnekler tamponda bekletilir.
İNİYOR (sarı)	Boru üst konumdan 20 mm'den fazla ayrıldı; stroke başladı.
TAMAMLANDI (kırmızı)	Boru alta ulaştı ya da 2 s boyunca 12 mm'den az oynadı (durdu). Sonra 2 s daha veri toplanır.

Hazırla düğmesi ve P0l: Stroke'tan önce boru üstte dururken "Hazırla"ya basılır. Program kesintisiz **2 saniyelik** sakin bir pencere arar: bu sürede basınç en fazla **3 mbar**, konum en fazla **12 mm** oynamalıdır. Bulunca bu penceredeki basınçların **ortancasını** (sıralayıp ortadakini) P0l olarak saklar. P0l = boru kıpırdamadan önceki bekleme basıncı, yani betonun kendi ağırlığıdır. Hazır olmadan stroke başlamaz.

Neden ortalama değil ortanca? Tek bir sıçrayan okuma ortalamayı kaydırır ama ortancayı pek etkilemez.

P0r: Stroke bittikten 0,5 s sonra başlayan 2 saniyedeki basınçların ortancası. Yalnızca bilgi olarak saklanır; hesapta P0l kullanılır.

> **Kodda nerede?**
>
> `Calculator.cpp → konumGuncelle()` (68) — durum makinesi.
> `Calculator.cpp → hazirlamaOrnegiIsle()` (350) — P0l penceresi.
> `Calculator.cpp → strokuBaslat() / strokuBitir()` (207 / 248).
> `SliperModel.cpp → durgunBasincMbar()` (168) — ortanca.
> Süreler ve toleranslar: `SliperModel.h` satır 47–63.

3.4 Bir stroke'tan iki sayı: p ve Q

Stroke basıncı p — "kaymaya karşı direnç"

Kayma sırasında basınç yükselir. En yüksek değer **Pmax**'tır; ama tek bir titreşim okumasını Pmax saymamak için her nokta etrafında **0,3 saniyelik** pencerenin ortalaması alınır (en az 3 örnek) ve bu ortalamaların en büyüğü Pmax olur.

$$p = P_{max} - P_{0l}$$

Örnek: P0l = 152,74 mbar, Pmax = 196,36 mbar → p = **43,62 mbar**. Betonun ağırlığını çıkarıyoruz ki geriye yalnızca "kaymaya karşı direnç" kalsın.

Hız v ve debi Q — "saatte ne kadar beton akardı"

Hız, yolun süreye bölümüdür. Program daha hassas yapar: stroke yolunun ilk %20'sini ve son %20'sini atar (hızlanma ve yavaşlama), ortadaki konum–zaman noktalarına en uygun doğruyu çizer; doğrunun eğimi hızdır. Ortada 3'ten az nokta varsa basitçe toplam yol ÷ toplam süre kullanılır.

$$Q\ (m^3/h) = v\ (m/s) \times \text{boru alanı}\ (m^2) \times 3600$$

Örnek: 0,5 m yol 4,64 s'de alınmış → v = 0,108 m/s → Q = 0,108 × 0,01287 × 3600 = **4,99 m³/h**. (×3600 saniyeyi saate çevirir.)

Kodda nerede?

`SliperModel.cpp → filtreliMaksimum()` (128) — Pmax.
`SliperModel.h → strokeBasinciMbar()` — p = Pmax − P0l.
`SliperModel.cpp → strokeHiziMs()` (96) — v.
`SliperModel.h → hizdanDebiM3h()` (42) — Q.
Hepsini sırayla çağıran: `Calculator.cpp → strokuBitir()` (248).

3.5 Hatalı stroke ve otomatik eleme

Şunlardan biri varsa stroke **"Wrong Stroke"** sayılır ve hesaba katılmaz:

Kod	Koşul	Anlamı
`kesinti`	İki örnek arası > 0,3 s	Wi-Fi verisi kesilmiş, eğri güvenilmez
`ornek`	İnişte 5'ten az örnek	Çok kısa, anlamlı hesap yapılamaz
`hiz`	v ≤ 0	Boru kaymamış veya takılmış
`basinc`	p ≤ 0	Kaymada basınç artmamış (P0l yanlış olabilir)

Takılma ile otomatik eleme: Geçerli stroke'larda bile boru yolun ortasında bir an takılıp sonra devam etmiş olabilir. Program her stroke için **takılma oranı** hesaplar: yolun %5–%80 arasındaki en düşük anlık hız ÷ stroke'un tipik (ortanca) hızı. Oran **0,15'ten küçükse** (yani boru bir an tipik hızının %15'inin altına düşmüşse) stroke otomatik hariç adayıdır. Güvenlik için toplamın en fazla %25'i ve her ağırlıktan en fazla 1 stroke elenir. Tabloda "Otomatik hariç" yazar; elle geri dahil edebilirsiniz.

Kodda nerede?

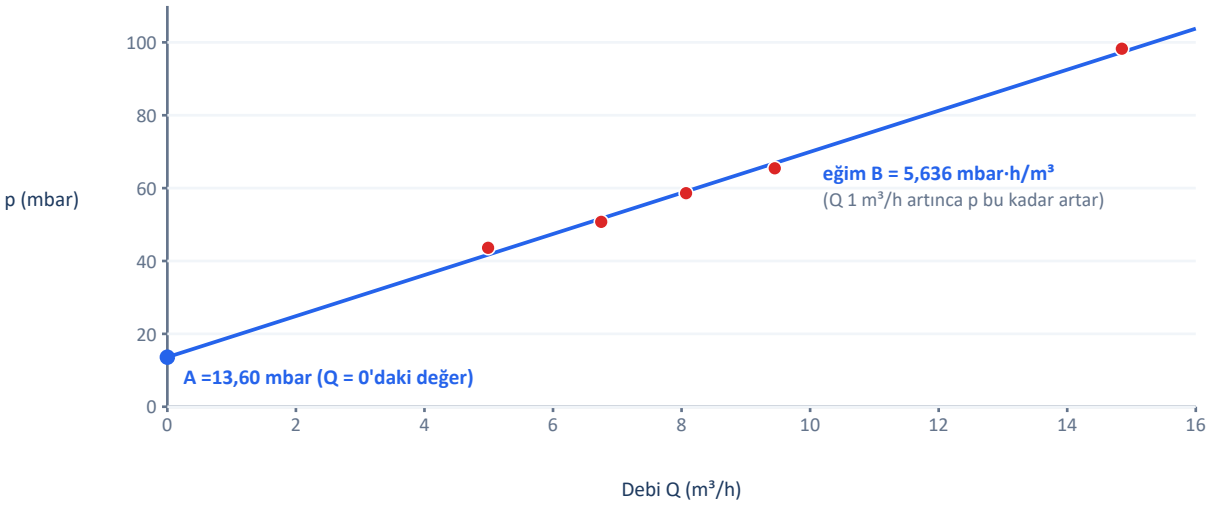
`SliperModel.cpp → strokeGecersizNedeni()` (190), `takilmaOrani()` (200), `otomatikElenecekler()` (240).
Uygulayan: `Database.cpp → otomatikElemeUygula()` (264).

3.6 Bütün stroke'lardan bir doğru: A, B ve R^2

Farklı ağırlıklarla stroke atınca her biri bir (Q, p) noktası verir. Ağırlık arttıkça boru hızlanır, p de artar. Bu noktalardan geçen en uygun düz çizgi çizilir (en küçük kareler yöntemi — Excel'deki "eğilim çizgisi"nin aynısı):

$$p = A + B \times Q$$

Stroke	1	2	3	4	5
Q (m^3/h)	4,99	6,75	8,07	9,45	14,85
p (mbar)	43,62	50,74	58,61	65,44	98,26



- **A = 13,60 mbar** — çizginin Q = 0'daki değeri: "harekete başlamak için gereken basınç". Akma direncinin ham hali.
- **B = 5,636 mbar·h/m³** — çizginin eğimi: "debi 1 m³/h artınca p ne kadar artıyor". Viskoz direncin ham hali.
- **R²** — noktalar çizgiye ne kadar oturuyor. 1,00 kusursuz; **0,85'in altı** şüpheli sayılır.

Nasıl hesaplanır? Önce Q'ların ortalaması (Q_{ort}) ve p'lerin ortalaması (p_{ort}) alınır. Sonra:

$$B = \Sigma (Q - Q_{ort}) \times (p - p_{ort}) \div \Sigma (Q - Q_{ort})^2$$
$$A = p_{ort} - B \times Q_{ort}$$

Σ "bütün stroke'lar için topla" demektir. Kısaca: Q ortalamadan sapınca p ne kadar sapıyor — bunun ortalama oranı eğimdir.

R² ayrıntısı: En az 3 farklı ağırlık kullanılmışsa R², her ağırlığın stroke'larının **ortalaması** alınarak hesaplanır. Aynı ağırlıktaki stroke'ların doğal dağılımı R²'yi haksız yere düşürmesin diye. A ve B ise yine tek tek bütün noktalardan hesaplanır.

Hangi stroke'lar katılır: geçerli olanlar **ve** tabloda "Dahil" işaretli olanlar.

> **Dikkat: isim karışıklığı**
>
> Veritabanında ve bazı ekran kodlarında A'ya **tau0**, B'ye **mu** denir. Bu eski bir isimlendirmedir; içerik A ve B'dir.

> **Kodda nerede?**
>
> `SliperModel.cpp → enUygunCizgi()` (20) — en küçük kareler.
> `SliperModel.cpp → pqDogrusu()` (265) — A, B, R² (ağırlık ortalamalı R²).
> `Database.cpp → binghamHesapla()` (469) — hangi stroke'lar katılacak.

3.7 Ölçüm kalite uyarıları

A ve B hesaplandıktan sonra ölçümün güvenilirliği kontrol edilir. Uyarılar kararı engellemez (ikisi hariç, bkz. 4.4), sadece haber verir:

Uyarı kodu	Ne zaman?	Ne yapmalı?
`agirlikTepkisiZayif`	En ağır ile en hafif yük arasındaki Pmax farkı, eklenen ağırlığın beklenen basıncının yarısından az	Load cell / P0l kontrolü
`azStroke`	Tahmine katılan stroke sayısı < 3	Daha fazla stroke atın

Uyarı kodu	Ne zaman?	Ne yapmalı?
`egimNegatif`	B < 0 (hız artınca basınç azalıyor — fiziksel değil)	Ölçümü tekrarlayın
`kesisimNegatif`	A < 0 (fiziksel değil)	Ölçümü tekrarlayın
`dusukR2`	$R^2 < 0{,}85$	Sapan stroke'ları hariç tutun / tekrar ölçün
`darDebiAraligi`	(Qmaks − Qmin) < ortalama Q'nun %30'u	Ağırlık farkını artırın
`tekYuk`	Tek bir ağırlıkla ölçülmüş	Ek ağırlıkla stroke atın
`yukBasinaAzStroke`	Bir ağırlıkta 3'ten az stroke	O ağırlıkla tekrar atın
`cokGecersiz`	Stroke'ların %30'undan fazlası hatalı	Cihaz / bağlantı kontrolü

Kodda nerede?

`SliperModel.cpp` → `olcumUyarilari()` (334), `agirlikTepkiOrani()` (313).
Eşikler: `SliperModel.h` satır 109–115 (`MIN_R2`, `MIN_DEBI_ARALIGI_ORANI` …).

4. Pompa basıncı tahmini

4.1 A ve B'yi boyuttan bağımsız yapmak: a ve b

A ve B bizim cihazın boyutlarına bağlıdır (128 mm çap, 500 mm dolum). Başka bir boruya taşımak için boyuttan arındırılır. Sonuç, Schleibinger cihazının gösterdiği "Yield Pressure a" ve "Pressure Gradient b" ile aynı şeydir ve doğrudan karşılaştırılabilir.

$$a = D \times A \div (4 \times l)$$
$$b = B \times \pi \times D^3 \div (16 \times l)$$

D = 0,128 m (cihaz borusu), l = 0,5 m (dolum yüksekliği).

Örnek: a = 0,128 × 13,60 ÷ 2 = **0,870 mbar**; b = 5,636 × π × 0,128³ ÷ 8 = 0,00464 → ekranda ×1000 ile **4,642** gösterilir (Schleibinger de öyle gösterir).

Neden bu formül? Boru duvarına düşen kesme gerilmesi = basınç × çap ÷ (4 × uzunluk). a, duvardaki akma gerilmesidir; b, duvardaki gerilmenin kayma hızıyla ne kadar arttığıdır. İkisi de yağlama tabakasının özelliğidir, cihazın değil.

Kodda nerede?

`SliperModel.h` → `schleibingerA(), schleibingerB()` (satır 123–131).

4.2 Gerçek pompa hattının basıncı

Sonuçlar ekranındaki **Tahmin Ayarları** kartına şantiyedeki hat bilgileri girilir:

Alan	Varsayılan	Anlamı
Hedef debi Q	30 m³/h	Pompanın basacağı debi
Boru iç çapı D	125 mm	Pompa hattının iç çapı
Hat uzunluğu	100 m	Düz boru toplam uzunluğu
90° / 45° dirsek	0 / 0	Her 90° dirsek 3 m, her 45° dirsek 1,5 m düz boru sayılır
Pompalama yüksekliği h	0 m	Betonun çıkacağı yükseklik (iniş varsa negatif)
Yoğunluk ρ	2400 kg/m³	Taze betonun yoğunluğu
Hata payı	%10	Sürtünme kısmına uygulanan ± belirsizlik
Pompa maks.	0 (girilmedi)	Pompanın verebileceği en yüksek basınç (bar)

$$\text{hesap uzunluğu } L = \text{hat uzunluğu} + 90^\circ \text{ dirsek sayısı} \times 3 + 45^\circ \text{ dirsek sayısı} \times 1{,}5$$

$$\text{akma payı} = 4 \times L \div D \times a$$
$$\text{hız payı} = 16 \times L \times Q \div (\pi \times D^3) \times b$$
$$\text{yükseklik payı} = \rho \times g \times h$$

$$P\,(\text{bar}) = (\text{akma payı} + \text{hız payı} + \text{yükseklik payı}) \div 1000$$

Q burada m³/h olarak girilir; b de mbar·h/m³ ölçeğinde olduğu için birimler kendiliğinden tutar. Yükseklik payı Pa çıkar, ÷100 ile mbar'a çevrilir.

Örnek: D = 126 mm, L = 50 m, Q = 10 m³/h, düz hat (h = 0):

- akma payı = 4 × 50 ÷ 0,126 × 0,870 = 1.382 mbar = **1,38 bar**
- hız payı = 16 × 50 × 10 ÷ (π × 0,126³) × 0,00464 = **5,91 bar**
- toplam = **7,29 bar**. Hat 10 m yukarı çıksaydı: + 2400 × 9,81 × 10 = **+2,35 bar**.

Bu formül Schleibinger kılavuzundaki tahmin tablolarını **birebir** verir (7,13 / 52,92 / 211,69 bar) — doğrulanmıştır.

Hata payı nasıl uygulanır?

Hata payı yalnızca **sürtünme kısmına** (akma + hız payı) uygulanır; yükseklik payı kesin fizik olduğu için uygulanmaz.

$$\text{alt sınır} = P - (\text{akma} + \text{hız}) \times \%\text{hata}$$
$$\text{üst sınır} = P + (\text{akma} + \text{hız}) \times \%\text{hata}$$

Pompa kararı her zaman **üst sınıra** göre verilir — yani temkinli tarafta kalınır.

Kodda nerede?

`SliperModel.cpp → basincTahmini()` (415) — tek bir debi/uzunluk için hesap.
`SliperModel.cpp → hesapUzunluguM()` (407) — dirsek eşdeğer uzunluğu.
`SliperModel.cpp → tahminTablosu()` (457) — ayarlarla satırı oluşturur, `pompaAsildi` bayrağını koyar.
`SliperModel.cpp → varsayilanTahminAyarlari()` (367) — varsayılanlar.
Dirsek değerleri: `SliperModel.h → DIRSEK_90_ESDEGER_M, DIRSEK_45_ESDEGER_M`.

4.3 Tahmin grafiği

Tahmin grafiğinde yatay eksen debi, dikey eksen basınçtır. Mavi çizgi gereken basınç (Q = 0'dan hedef debinin 1,5 katına kadar), gri kesikli çizgiler alt/üst sınır, kırmızı nokta ve çubuk hedef debideki tahmin, turuncu kesik-noktalı yatay çizgi pompa kapasitesidir. Açık yeşil "uygun bölge", üst sınırın pompa çizgisinin altında kaldığı debilerdir — yani bu hatla en fazla hangi debiye çıkabileceğinizi gösterir.

Kodda nerede?

`SonuclarPage.qml → tahminGrafiginiCiz()` (satır 239).

4.4 Karar: pompalanabilir mi?

Sonuçlar ekranındaki renkli durum kartı şu sırayla karar verir:

Sıra	Koşul	Kart	Kod
1	Tahmine katılan stroke < 2	gri — Yeterli stroke yok	`yetersiz`
2	B < 0 veya A < 0	gri — Karar verilemez, ölçümü tekrarla	`gecersizModel`
3	Pompa maks. girilmemiş	gri — Değerlendirilmedi	`yok`
4	Üst sınır > pompa maks.	**kırmızı — POMPALAMA SORUNU** + reçete önerisi	`sorun`
5	Pompa yetiyor ama $R^2 < 0,85$	turuncu — Pompalanabilir, güvenilir değil	`guvenilmez`
6	Pompa yetiyor, R^2 iyi	**yeşil — POMPALANABİLİR** (kapasite kullanım yüzdesiyle)	`uygun`

Dikkat: düşük R^2 (sıra 5) pompa aşılıyorsa kararı değiştirmez — pompa aşılıyorsa kart yine kırmızıdır.

Kodda nerede?

`SonuclarPage.qml → durumBilgisi` (satır 284).
`SliperModel.cpp → tahminGuvenilmezNedeni(), tahminKarariVerilemez()` (474–485).
Rapordaki karşılığı: `ReportManager.cpp` satır ~370–415.

5. Reçete önerisi: "akışkanlaştırıcı / kum / agrega" nereden geliyor?

Bu bölüm, sorduğunuz asıl konudur. Kısaca: program **reçete miktarı hesaplamaz**. Yalnızca pompa yetmediğinde, gereken basıncın **en büyük kısmının nereden geldiğine** bakar ve buna göre **yön** söyler (artırın / azaltın).

5.1 Öneri ne zaman çıkar?

Yalnızca durum kartı **kırmızı (POMPALAMA SORUNU)** olduğunda. Yani:

- Yeterli stroke var, A ve B pozitif (model geçerli),
- Pompa maks. basıncı girilmiş,
- Tahminin **üst sınırı** pompa maks. basıncını aşıyor.

Pompa yetiyorsa öneri gösterilmez — çalışan reçeteyi değiştirmeye gerek yoktur.

5.2 Karar mantığı (üç dal)

Program tahmin satırındaki üç payı (akma, hız, yükseklik) karşılaştırır. Kodun tamamı şudur:

```
QString pompaSorunuNedeni(const QVariantMap &tahminSatiri, double pompaMaksBar)
{
    const double yukseklikBar = tahminSatiri["yukseklikMbar"].toDouble() / 1000.0;
    if (yukseklikBar >= pompaMaksBar) return "yukseklik";
    if (tahminSatiri["viskozMbar"].toDouble() > tahminSatiri["akmaMbar"].toDouble())
        return "viskoz";
    return "akma";
}
```

Türkçesiyle:

Soru	Cevap evetse	Ekrandaki öneri
1) Yalnızca **yükseklik payı** bile pompa kapasitesine ulaşıyor mu?	**"yukseklik"** — Reçeteyle çözülmez	• Daha yüksek basınçlı pompa gerekir
2) **Hız payı > akma payı** mı?	**"viskoz"** — Karışım yapışkan	• Katkıyı (akışkanlaştırıcı) artırın • Viskozite düzenleyici katkıyı azaltın • Toz / ince malzemeyi azaltın
3) Değilse (akma payı ≥ hız payı)	**"akma"** — Karışım sert	• Katkıyı (akışkanlaştırıcı) artırın • İnce kum / harç oranını artırın • İri agregayı azaltın

Viskoz ve akma dallarında altta **"Değişiklikten sonra tekrar ölçün."** notu çıkar; yükseklik dalında çıkmaz (reçete değişmeyeceği için).

5.3 Her önerinin arkasındaki fizik

Dal 1 — Yükseklik: "Reçeteyle çözülmez"

Yükseklik payı ρ × g × h'dir: betonun ağırlığını h metre yukarı kaldırmanın bedeli. Bu, betonun kıvamına değil yalnızca **yoğunluğuna** ve **yüksekliğe** bağlıdır. Sürtünmeyi sıfıra indirseniz bile bu pay kalır. Tek başına pompa kapasitesini dolduruyorsa reçeteyle yapılacak bir şey yoktur; daha güçlü pompa gerekir.

Dal 2 — Viskoz baskın: "Karışım yapışkan"

Basıncın çoğu **b**'den geliyor: duvardaki yağlama tabakası koyu/yapışkan, hız arttıkça direnç hızla büyüyor (bal benzetmesi). Öneriler bu tabakayı "incelten" yöndedir:

- **Akışkanlaştırıcıyı artırın** — çimento tanelerini birbirinden ayırır, hamuru daha akıcı yapar.
- **Viskozite düzenleyici katkıyı (VMA) azaltın** — VMA'nın işi zaten betonu koyulaştırmaktır (ayrışmayı önlemek için); fazlası yapışkanlık demektir.
- **Toz / ince malzemeyi azaltın** — çok fazla ince malzeme (filler, mikrosilis, çok ince kum) su ihtiyacını ve hamurun koyuluğunu artırır.

Dal 3 — Akma baskın: "Karışım sert"

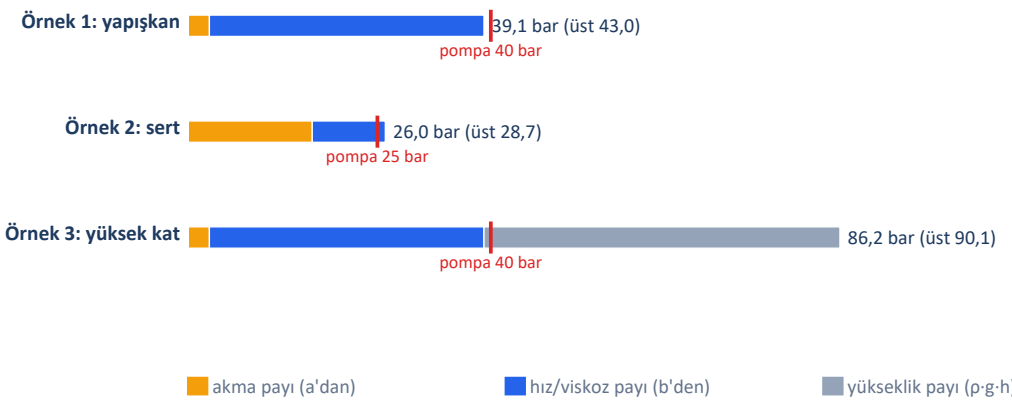
Basıncın çoğu **a**'dan geliyor: betonu harekete geçirmek zor. Genelde boru duvarında yeterli **yağlama tabakası oluşturacak harç yoktur**; iri taneler duvara sürtünür (ketçap / kızak benzetmesi). Öneriler yağlama tabakasını "besleyen" yöndedir:

- **Akışkanlaştırıcıyı artırın** — akma eşiğini düşürmenin en hızlı yolu.
- **İnce kum / harç oranını artırın** — duvarda kayma tabakasını oluşturacak ince malzeme artar.
- **İri agregayı azaltın** — iri taneler (çakıl) duvara temas edip sürtünür; azalınca kayma kolaylaşır.

> **Neden bir yerde "ince malzemeyi azalt", diğerinde "ince kumu artır"?**
>
> Dikkat edin: **Dal 2 "inceyi azalt", Dal 3 "inceyi artır"** der. Çelişki değildir. Yapışkan karışımda ince malzeme fazladır; sert karışımda azdır. Doğru yönü ölçüm söyler — bu yüzden önerinin kendisi ölçüme bağlıdır.

5.4 Üç çalışılmış örnek

Hepsinde hat: D = 125 mm, L = 100 m, Q = 30 m³/h, ρ = 2400, hata payı %10. Örnek 1 ve 3 yukarıdaki ölçümden (A = 13,60, B = 5,636); Örnek 2 uydurma bir sert beton (A = 80, B = 1,5).



Örnek	Akma bar	Hız bar	Yüks. bar	P / üst bar	Pompa	Dal	Öneri
1	2,79	36,31	0,00	39,09 / **43,00**	40	viskoz	Karışım yapışkan
2	16,38	9,66	0,00	26,05 / **28,65**	25	akma	Karışım sert
3 (h = 200 m)	2,79	36,31	47,07	86,17 / **90,08**	40	yukseklik	Reçeteyle çözülmez

Örnek 1: Üst sınır 43,00 bar > 40 bar → sorun. Yükseklik 0, pompa aşılmıyor; hız payı (36,31) > akma payı (2,79) → **yapışkan**.

Örnek 2: Üst sınır 28,65 bar > 25 bar → sorun. Akma payı (16,38) ≥ hız payı (9,66) → **sert**.

Örnek 3: Yükseklik payı = 2400 × 9,81 × 200 = 47,07 bar ≥ 40 bar → sürtünmeye bakılmadan **reçeteyle çözülmez**.

5.5 Bilinmesi gereken sınırlar

- **Miktar söylemez.** "%0,2 akışkanlaştırıcı ekleyin" gibi bir sayı yoktur; çünkü etkisi çimentoya, katkının markasına, agregaya bağlıdır. Değişiklik yapılır, **tekrar ölçülür**.
- **Karşılaştırma hedef debide yapılır.** Hız payı debiyle büyür. Aynı beton düşük debide "sert", yüksek debide "yapışkan" çıkabilir. Bu bir hata değil: o debide hangi direncin baskın olduğunu söyler.
- **Hat ve debi önerilmez.** Daha geniş boru, daha kısa hat veya daha düşük debi de basıncı düşürür; ama bunlar şantiyede belirlendiği için program önermez. (Tahmin grafiğindeki yeşil "uygun bölge" hangi debiye kadar inilebileceğini gösterir.)
- **Sınır durum:** Akma ve hız payı eşitse "sert" dalı seçilir.
- **Conta sürtünmesi:** Bizim cihazda load cell piston altında olduğu için conta sürtünmesi de ölçülür; A (dolayısıyla akma payı) biraz yüksek çıkabilir. Bu da kararı "sert" yönüne itebilir. Boş borulu kör test henüz yapılmadı.

5.6 Öneri kodun neresinde?

Dosya / yer	Ne yapar
`SliperModel.cpp → pompaSorunuNedeni()` (487)	Üç dallı kararı verir: "yukseklik" / "viskoz" / "akma". Tek doğruluk kaynağı.
`Calculator.cpp → pompaSorunuNedeni()` (494)	Aynı fonksiyonu QML ekranına açan köprü (Q_INVOKABLE).
`SonuclarPage.qml → durumBilgisi` (284)	Pompa aşıldı mı bakar, en kötü satırı bulur, sorunNedeni'ni alır.
`SonuclarPage.qml → receteOnerisi` (301)	sorunNedeni'ne göre başlığı ve madde listesini seçer.
`SonuclarPage.qml → metinler` (66–75)	Öneri cümlelerinin Türkçe/İngilizce metinleri (oneriKatkiArtir, oneriInceKumArtir …).
`SonuclarPage.qml` (~664–690)	Öneri kutusunu ekrana çizer.
`ReportManager.cpp` (387–413)	PDF rapora "Reçete Önerisi" bölümünü gerekçe cümlesiyle birlikte yazar.

Değişiklik ipucu

Öneri cümlesini değiştirmek isterseniz **iki yerde** değiştirin: ekran için `SonuclarPage.qml → metinler`, rapor için `ReportManager.cpp` satır 393–408. Karar kuralını değiştirmek isterseniz yalnızca `SliperModel.cpp → pompaSorunuNedeni()` yeterlidir; ekran ve rapor ikisi de onu çağırır.

6. Kod haritası

6.1 Dosyalar

Dosya	Ne işe yarar	Ne zaman açarsınız
`src/SliperModel.h/.cpp`	**Bütün formüller ve ayar sayıları**: cihaz ölçüleri, kalibrasyon, stroke süreleri, uyarı eşikleri, tahmin, öneri kararı	Bir hesabı merak ettiğinizde
`src/WifiManager.cpp/.h`	ESP32'den veri alır; kg/mm/basınca çevirir; dara; bağlantı	Bağlantı / sensör sorunu
`src/Calculator.cpp/.h`	Stroke'u yakalar (durum makinesi), Hazırla/P0l, stroke sonunda p ve Q; SliperModel'i QML'e açar	Stroke algılanmıyorsa
`src/Database.cpp/.h`	SQLite kayıt; A–B için stroke seçimi; otomatik eleme; Excel (XML) / CSV	Kayıt, dışa aktarma
`src/ReportManager.cpp/.h`	PDF rapor, grafikler, durum ve reçete önerisi	Rapor görünümü
`src/SensorManager.cpp/.h`	Anlık değerleri ekrana taşıyan ara katman	Nadiren
`src/VoiceCommandManager / VoiceRecognitionWorker`	Sesli komut (Vosk, internetsiz)	Sesli komut
`src/Backend.cpp/.h`	Giriş ekranı kullanıcı/şifre	Şifre değişikliği
`main.cpp`	Programı başlatır, C++ sınıflarını QML'e tanıtır	Yeni sınıf eklenince
`qml/pages/OlcumPage.qml`	Ölçüm ekranı (canlı grafik, Hazırla, ağırlık ekle)	Ölçüm ekranı
`qml/pages/SonuclarPage.qml`	Sonuçlar, P–Q grafiği, tahmin, durum kartı, reçete önerisi	Sonuç ekranı
`qml/pages/KalibrasyonPage.qml`	Load cell, mesafe, eğim, stroke konumları	Kalibrasyon
`qml/pages/GecmisPage.qml`	Eski ölçümler, yedekleme	Geçmiş
`esp32/sliper_esp32/sliper_esp32.ino`	ESP32 yazılımı (sensör okuma, Wi-Fi)	Donanım/pin

6.2 SliperModel fonksiyonları tek tabloda

Fonksiyon	Girdi → Çıktı	Bölüm
`kalibrasyonTablosundanOku`	ham sayı → kg veya mm	3.1
`konumFiltrele / agirlikFiltrele`	önceki + yeni → süzülmüş değer	3.1
`agirliktanBasincMbar`	kg → mbar	3.2
`durgunBasincMbar`	basınç listesi → ortanca (P0l, P0r)	3.3
`filtreliMaksimum`	zaman + basınç → Pmax (0,3 s ortalamalı)	3.4
`strokeBasinciMbar`	Pmax, P0l → p	3.4
`strokeHiziMs`	zaman + konum → v (orta %60'a doğru)	3.4
`hizdanDebiM3h`	v → Q	3.4
`strokeGecersizNedeni`	örnek sayısı, boşluk, v, p → hata kodu	3.5
`takilmaOrani / otomatikElenecekler`	konum eğrisi → takılma; eleme listesi	3.5
`enUygunCizgi / pqDogrusu`	(Q, p) noktaları → A, B, R^2	3.6
`olcumUyarilari / agirlikTepkiOrani`	A, B, R^2, sayılar → uyarı listesi	3.7
`schleibingerA / schleibingerB`	A → a, B → b	4.1

Fonksiyon	Girdi → Çıktı	Bölüm
`hesapUzunluguM`	hat + dirsekler → L	4.2
`basincTahmini`	A, B, Q, D, L, h, ρ, hata → P, alt, üst, üç pay	4.2
`tahminTablosu`	A, B, ayarlar → tahmin satırı + pompaAsildi	4.2
`tahminGuvenilmezNedeni` / `tahminKarariVerilemez`	A, B, R^2 → güvenilirlik	4.4
`pompaSorunuNedeni`	tahmin satırı, pompa maks → yukseklik / viskoz / akma	5
`hizEgrisiMs`	zaman + konum → anlık hız eğrisi (0,4 s pencere)	grafik

7. Ayar sayıları (SliperModel.h)

Bu sayıları değiştirmek programın davranışını değiştirir. Değiştirmeden önce neden o değerde olduklarını bilin; çoğu sahada denenerek seçilmiştir.

Sabit	Değer	Ne işe yarar
`BORU_CAPI_M`	0,128	Cihaz borusu iç çapı (m) — boru değişirse burası
`BORU_UZUNLUGU_M`	0,5	Dolum yüksekliği l (m)
`KONUM / AGIRLIK_FILTRE_KATSAYISI`	0,35 / 0,4	Süzgeçte yeni okumanın payı
`DARA_UST_SINIRI_KG`	2	Dara en fazla bu kadar olabilir
`UST / ALT_YAKALAMA_TOLERANSI_MM`	20 / 25	Üstten ayrılma / alta varma payı
`DURGUN_TOLERANS_MM`	12	Hazırla'da konum en fazla bu kadar oynayabilir
`HAZIRLAMA_SURESI_S / _BASINC_TOLERANSI_MBAR`	2 / 3	P0l için sakin pencere
`DURMA_TOLERANSI_MM / DURMA_ONAYI_S`	12 / 2	Boru durdu sayılması için
`ONCESI / SONRASI_PENCERE_S`	2 / 2	Stroke öncesi/sonrası saklanan süre
`P0R_OTURMA_S`	0,5	P0r'den önce beklenen süre
`HIZ_UYUM_ALT / UST`	0,2 / 0,8	Hız hesabında kullanılan yol aralığı
`PMAKS_PENCERE_S / PMAKS_MIN_ORNEK`	0,30 / 3	Pmax ortalama penceresi
`MIN_STROKE_ORNEGI / MAKS_ORNEK_ARALIGI_S`	5 / 0,3	Hatalı stroke sınırları
`TAKILMA_ESIGI / MAKS_OTOMATIK_ELEME_YUZDE`	0,15 / 25	Otomatik eleme
`MIN_R2`	0,85	R^2 bunun altındaysa "güvenilir değil"
`MIN_DEBI_ARALIGI_ORANI`	0,3	Dar debi aralığı uyarısı
`MIN_YUK_BASINA_STROKE / MAKS_GECERSIZ_YUZDE`	3 / 30	Stroke sayısı uyarıları
`MIN_AGIRLIK_TEPKI_ORANI`	0,5	Ağırlık tepkisi zayıf uyarısı
`DIRSEK_90 / 45_ESDEGER_M`	3 / 1,5	Dirseklerin düz boru karşılığı (m)
`HIZ_PENCERESI_S`	0,40	Canlı hız ve hız eğrisi penceresi

Sözlük

Terim	Anlamı
P0l	Stroke öncesi bekleme basıncı (betonun kendi ağırlığı)
P0r	Stroke sonrası bekleme basıncı
Pmax	Stroke sırasındaki en yüksek (0,3 s ortalamalı) basınç

Terim	Anlamı
p	Stroke basıncı = Pmax − P0l
v, Q	Kayma hızı (m/s), debi (m^3/h)
A, B	p–Q doğrusunun başlangıcı (mbar) ve eğimi ($mbar \cdot h/m^3$) — cihaza bağlı
a, b	A ve B'nin boyuttan bağımsız hali — Schleibinger "Yield Pressure" ve "Pressure Gradient"
R^2	Noktaların doğruya uyumu (1 = mükemmel)
Akma payı / hız payı / yükseklik payı	Gereken pompa basıncının üç parçası
VMA	Viskozite düzenleyici katkı (betonu koyulaştırır, ayrışmayı önler)
Akışkanlaştırıcı	Süper akışkanlaştırıcı katkı (betonu akıcılaştırır)